

PROJECT DEVELOPMENT PHASE

1. Creating Project Folder

Steps to support the backend project structure:

1. Create the root project folder, e.g. AI-FitTrack-Backend, and open it in Visual Studio Code.
2. Inside it, create a dedicated server/ directory to hold all backend source code.
3. Inside server/, create the sub-folders: config/, controllers/, middleware/, models/, routes/, and services/.

2. Server Setup

Run the following terminal commands to initialize the Node.js environment and install required packages:

```
# 1. Initialize package.json
npm init -y

# 2. Install production dependencies
npm install express mongoose jsonwebtoken bcryptjs @google/genai dotenv cors

# 3. Install development dependency for live server auto-reload
npm install -D nodemon
```

The application entry point (server.js) wires up Express, CORS, JSON body parsing, the database connection, and every route group.

3. Backend Structure

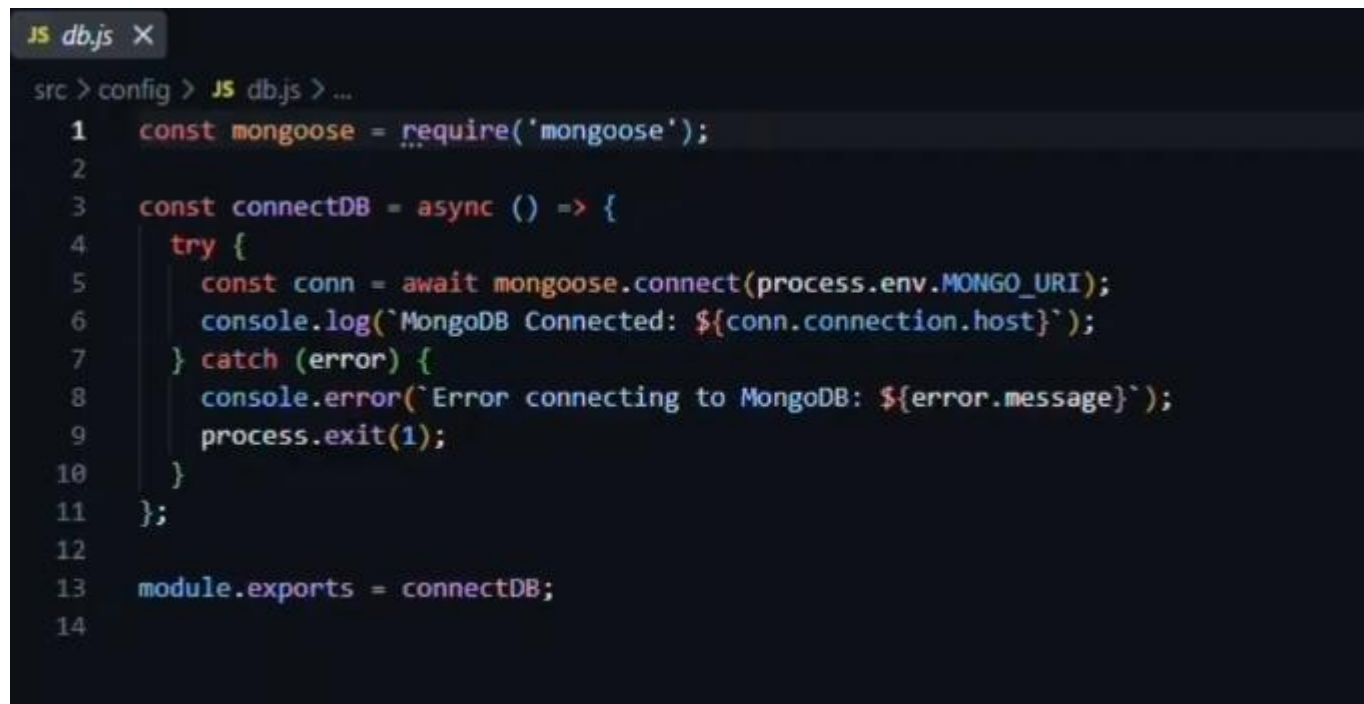
The GitHub repository organizes backend source code inside a dedicated /server directory to maintain modular separation:

```
AI-FitTrack-Backend/
|-- server/
|   |-- config/      # Database configuration (db.js)
|   |-- controllers/ # Auth, Workout, and AI business logic
|   |-- middleware/  # JWT protection and error middleware
|   |-- models/      # User and Workout Mongoose schemas
|   |-- routes/      # API route definitions
|   |-- services/     # Google Gemini AI wrapper module
|   |-- .env.example # Environment variable template
|   |-- package.json # Node.js dependencies and run scripts
|   |-- server.js     # Server application entry point
|-- .gitignore        # Git file exclusion rules
-- README.md         # Master setup guide & API reference
```

4. Development and Explanation

A. Configure MongoDB & Create Database Connection (config/db.js)

Connects the Express application to a MongoDB Atlas cluster (or a local MongoDB instance) using Mongoose. The MONGO_URI is generated from the MongoDB Atlas dashboard (Database → Connect → Drivers) and stored in .env, never committed to source control.



```
JS db.js X
src > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    try {
5      const conn = await mongoose.connect(process.env.MONGO_URI);
6      console.log(`MongoDB Connected: ${conn.connection.host}`);
7    } catch (error) {
8      console.error(`Error connecting to MongoDB: ${error.message}`);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
14
```

B. Create Schema and Models

The User and Workout Mongoose schemas (shown in full under FR1 and FR2 in the Requirement Analysis Phase) define the shape, types, and validation rules for both collections. They are the only layer that communicates directly with MongoDB.

C. Security & Authentication Middleware (middleware/authMiddleware.js)

Verifies incoming JWT tokens in the HTTP Authorization header to secure private endpoints.

```
JS auth.js X
src > middleware > JS auth.js > ...
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const protect = async (req, res, next) => {
5    let token;
6
7    if (
8      req.headers.authorization &&
9      req.headers.authorization.startsWith('Bearer')
10   ) {
11     try {
12       // Get token from header
13       token = req.headers.authorization.split(' ')[1];
14
15       // Verify token
16       const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18       // Get user from the token, excluding password
19       req.user = await User.findById(decoded.id).select('-password');
20
21       if (!req.user) {
22         return res.status(401).json({
23           success: false,
24           error: 'Not authorized, user not found',
25         });
26       }
27
28       return next();
29     } catch (error) {
30       return res.status(401).json({
31         success: false,
32         error: 'Token is not valid',
33       });
34     }
35   }
36
37   return next();
38 }
```

```

JS auth.js X
src > middleware > JS auth.js > ...
4   const protect = async (req, res, next) => {
13   // Get token from request
14   token = req.headers.authorization.split(' ')[1];
15   // Verify token
16   const decoded = jwt.verify(token, process.env.JWT_SECRET);
17   // Get user from the token, excluding password
18   req.user = await User.findById(decoded.id).select('-password');
21   if (!req.user) {
22     return res.status(401).json({
23       success: false,
24       error: 'Not authorized, user not found',
25     });
26   }
27
28   return next();
29 } catch (error) {
30   console.error('JWT verification error:', error.message);
31   return res.status(401).json({
32     success: false,
33     error: 'Not authorized, token failed',
34   });
35 }
36 }
37
38 if (!token) {
39   return res.status(401).json({

```

D. CRUD Operations Logic (controllers/workoutController.js)

Executes complete workout data management (Create, Read, Search/Filter, Update, Delete).

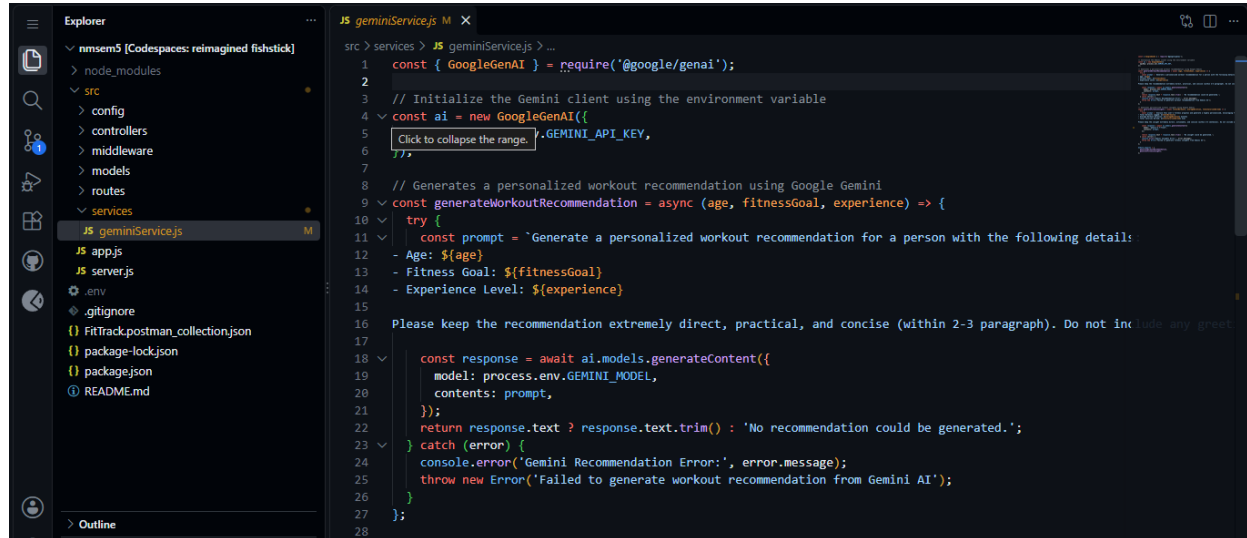
```

JS workoutController.js X
src > controllers > JS workoutController.js > ...
6   const addWorkout = async (req, res, next) => {
9
10   // Validate fields
11   if (!workoutName || !category || !duration || !caloriesBurned) {
12     return res.status(400).json({
13       success: false,
14       error: 'Please provide workoutName, category, duration, and caloriesBurned',
15     });
16   }
17
18   // Create workout record scoped to user
19   const workout = await Workout.create({
20     workoutName,
21     category,
22     duration,
23     caloriesBurned,
24     workoutDate: workoutDate || new Date(),
25     user: req.user._id,
26   });
27
28   res.status(201).json({
29     success: true,
30     data: workout,
31   });
32 } catch (error) {
33   next(error);
34 }
35 }

```

E. Gemini AI Integration Service (services/geminiService.js)

Constructs structured prompt payloads and extracts text outputs from @google/genai.



```
src > services > JS geminiService.js > ...
1  const { GoogleGenAI } = require('@google/genai');
2
3  // Initialize the Gemini client using the environment variable
4  const ai = new GoogleGenAI({
5    .GEMINI_API_KEY,
6  });
7
8  // Generates a personalized workout recommendation using Google Gemini
9  const generateWorkoutRecommendation = async (age, fitnessGoal, experience) => {
10   try {
11     const prompt = `Generate a personalized workout recommendation for a person with the following details:
12     - Age: ${age}
13     - Fitness Goal: ${fitnessGoal}
14     - Experience Level: ${experience}
15
16     Please keep the recommendation extremely direct, practical, and concise (within 2-3 paragraph). Do not include any greet
17
18     const response = await ai.models.generateContent({
19       model: process.env.GEMINI_MODEL,
20       contents: prompt,
21     });
22     return response.text ? response.text.trim() : 'No recommendation could be generated.';
23   } catch (error) {
24     console.error('Gemini Recommendation Error:', error.message);
25     throw new Error('Failed to generate workout recommendation from Gemini AI');
26   }
27 };
28
```

Fig. Gemini AI service implementation (services/geminiService.js)